

Lecture 1 - Introduction

CPE334 – Software Engineering (in the Age of AI Coding Agents)

Week#1: 3-7 August 2026

Dr. Suthep Madarasmi
Dr. Piyanit Ua-areemitr
Dr. Santawat Thanyadit

Department of Computer Engineering
King Mongkut's University of Technology Thonburi



Today's lecture storyline

From AI can write code to we can engineer a product.

- 1 **AI coding agents**
- 2 **Code vs software product**
- 3 **Software Development Life Cycle**
- 4 **Process models**
- 5 **Responsible AI use**
- 6 **Lab preparation**

AI coding agents

Not just autocomplete: agents can plan steps, call tools, edit files, run commands, and propose changes.

- A basic chatbot - mainly produces a response based on **inputs**
- AI agent – an **autonomous** software system that can pursue a **goal**
 - Understanding its **environment** and instructions
 - **Planning** a sequence of actions
 - **Using tools** or external systems
 - Observing the results
 - Adjusting its actions until the task is completed (**feedback loops**)
 - Retain context across interactions (**memory**)
- AI coding agent - performs software-development actions using code, tools, tests, and repositories





Is knowing how to use AI coding agents enough to develop software?



Which processes are necessary for software development?

Which processes are necessary for software development?

Software development is a sequence of decisions, not only a sequence of prompts.



- AI can assist at every stage, but every stage still needs a human accountable for the decision.
- A good software engineer learns how to convert messy needs into small, verified product increments.

Writing code vs engineering a software product

AI lowers the cost of code. It does not remove the cost of building the right thing well.

Writing code

- ✓ Implements an instruction
- ✓ Works on a local example
- ✓ May ignore future maintainers
- ✓ Can be generated quickly

Engineering a software product

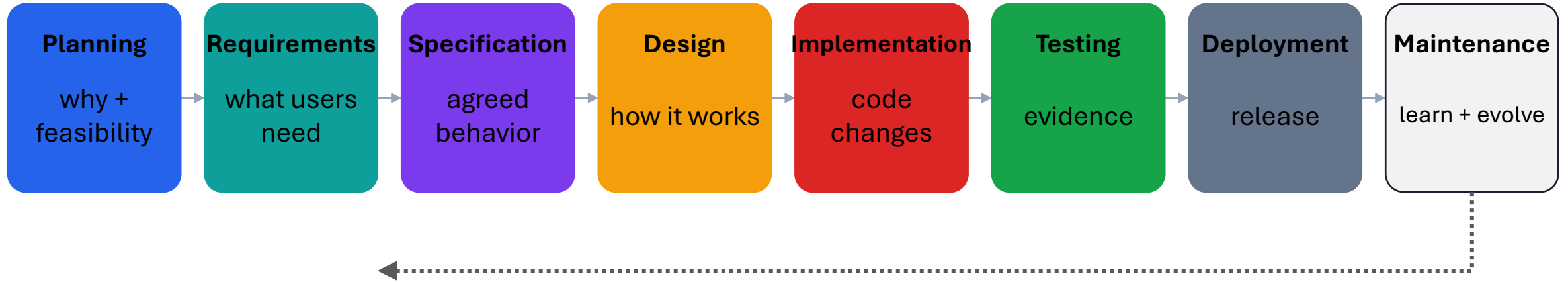
- ✓ Solves a user problem
- ✓ Meets explicit acceptance criteria
- ✓ Is testable, secure, maintainable
- ✓ Can be operated and evolved

Course principle

Use AI to accelerate implementation, but use software engineering to define, constrain, verify, and evolve the product.

Software Development Life Cycle (SDLC)

A repeatable roadmap for moving from a problem to a maintained product.



- Feedback loops: requirements change, tests reveal defects, deployment creates new learning.
- Different models arrange these phases differently; the activities never disappear.

SDLC phase 1–2: Planning and requirements

Before asking an agent to code, decide the product problem and the expected value.

Planning

- Stakeholders and users
- Goals and success measures
- Feasibility, constraints, risk
- Scope and priorities

Requirements

- Functional requirements: what the system must do
- Non-functional requirements: quality attributes
- Constraints: laws, platforms, budget, data
- Traceability: where each need came from

Requirement = a documented need, capability, or constraint that the software must satisfy.

SDLC phase 3: Specification as an engineering contract

A specification turns requirements into testable, reviewable agreement.

Specification

Precise description of expected behavior, interfaces, rules, data, errors, and quality constraints.

Acceptance criterion

A condition that must be true for a feature/user story to be accepted by the product owner or stakeholder.

Engineering contract

The shared baseline between stakeholders and engineers: what will be built, what counts as done, and how it will be verified.

Example

Requirement: Students can submit lab work online.

Specification: System accepts .ipynb files ≤ 25 MB before deadline; records timestamp; shows confirmation.

Acceptance criteria: Given a valid file before deadline, when the student clicks Submit, then the submission is stored and a confirmation ID is displayed.

Key idea: an AI coding agent should receive the specification, not just a vague wish.

SDLC phase 4: Design and architecture

Design decides the structure before code multiplies the cost of a bad decision.



Design artifacts

Architecture diagrams · data models · API/interface contracts · UI flows · security boundaries · error handling

AI can propose designs. Engineers must check trade-offs, consistency, scalability, security, and maintainability.

SDLC phase 5: Implementation with coding agents

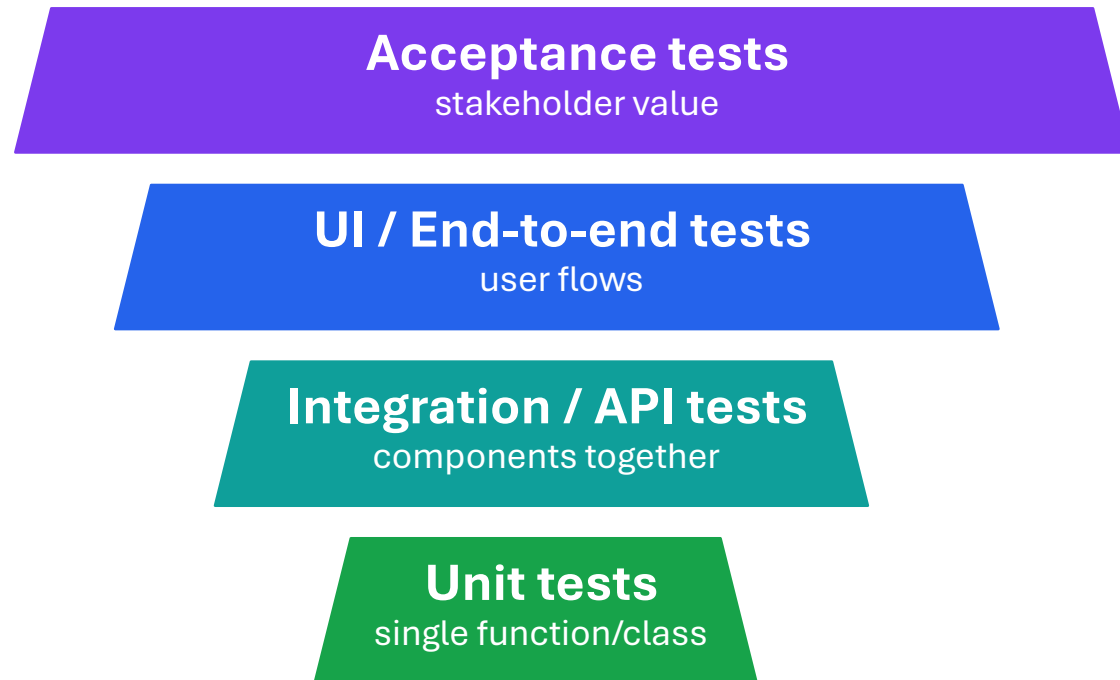
Implementation is where coding agents are strongest — when the task is small and well-specified.

Good delegation pattern

- 1 Give context** repo structure, files, constraints, coding standard
- 2 Give specification** inputs, outputs, edge cases, acceptance criteria
- 3 Ask for plan first** let the agent explain intended files and tests
- 4 Scope the increment** one feature, one bug, or one refactor at a time
- 5 Review evidence** diff, commands run, tests, docs, and residual risks

SDLC phase 6: Testing and verification

Testing is evidence. It does not prove perfection, but it reduces uncertainty.



Also test:

Regression: old behavior still works

Exploratory: human discovery of unexpected defects

Performance: speed, load, resource use

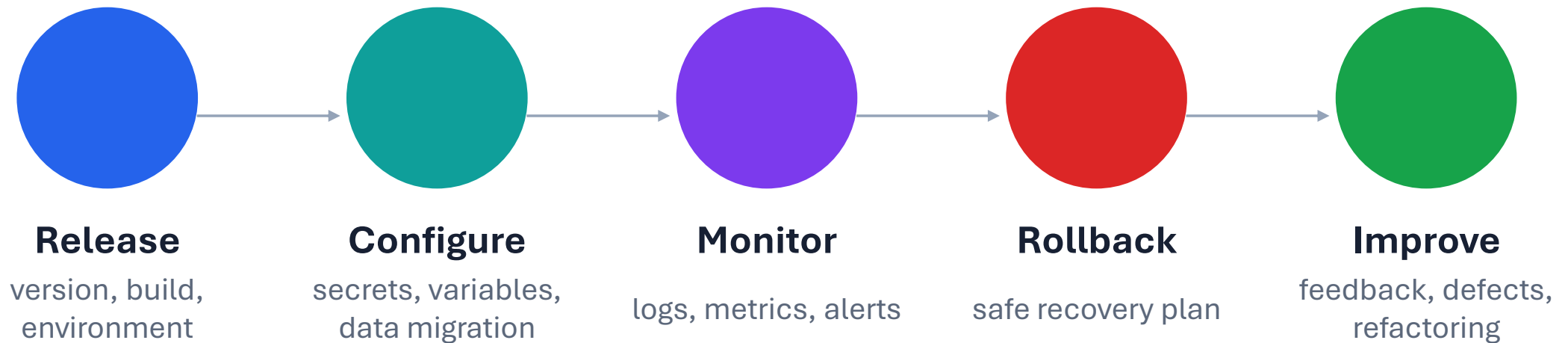
Security: misuse, secrets, injection, permissions

Usability: can real users complete the task?

AI-generated tests are useful but not automatically correct; check that tests would fail on a real defect.

SDLC phase 7–8: Deployment, operation, and maintenance

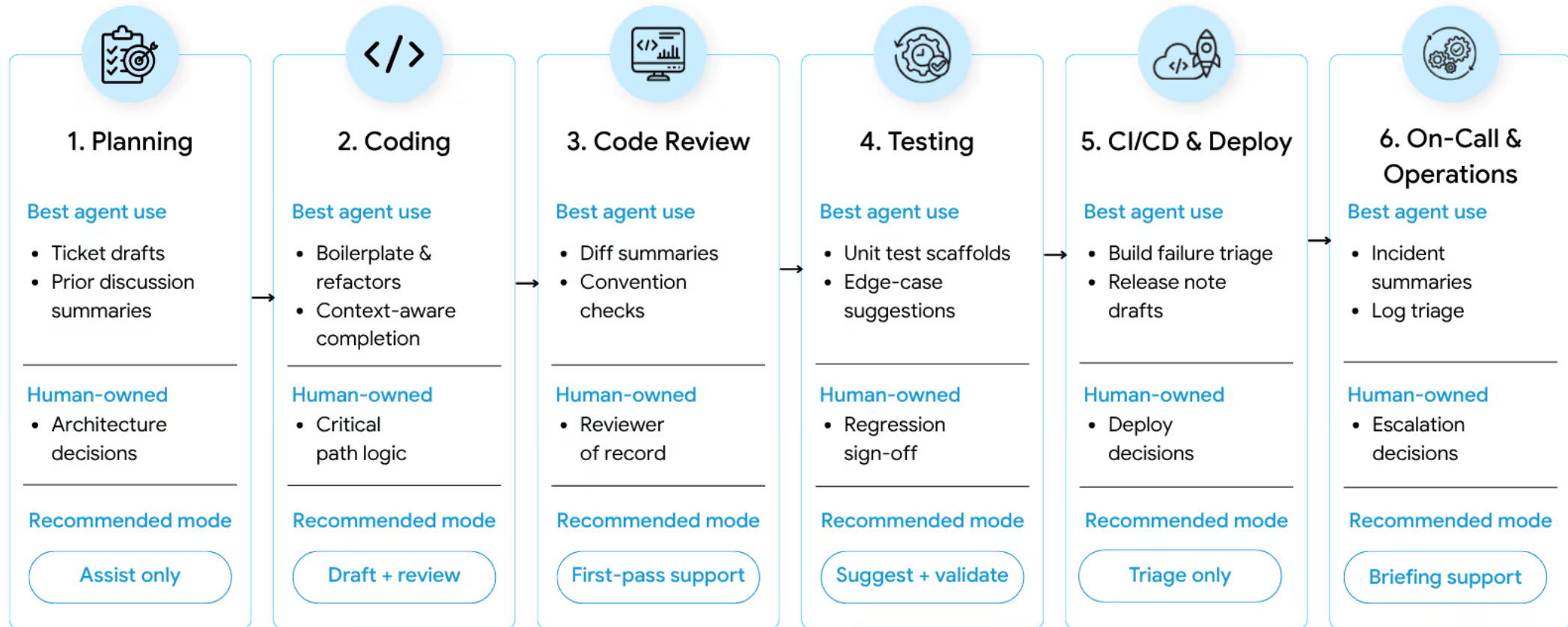
Software becomes real when users depend on it.



Role of AI coding agents across the SDLC

AI Agent Readiness Across the SDLC

Where agents create leverage, where they assist, and where human judgment stays in control



Use agents where context is available. Keep humans accountable where judgment is required.

Role of AI coding agents across the SDLC

Use the right agent behavior at the right phase.

Specification agent

clarify requirements, draft user stories, refine acceptance criteria, detect ambiguity

Coding agent

implement small scoped increments, refactor, update tests, generate docs

Debugging agent

reproduce defect, inspect logs, isolate root cause, propose fix and regression test

Never let the agent choose the product goal, acceptance standard, risk appetite, or final responsibility.

Risk: blindly accepting works

An AI agent's claim is not evidence.



Wrong target

implements a different requirement



Hidden breakage

passes examples but fails edge cases



Dependency risk

adds packages or commands you do not understand



False confidence

claims tests passed but did not run them



Security gaps

leaks secrets, weak permissions, unsafe commands



Documentation drift

README/comments no longer match behavior

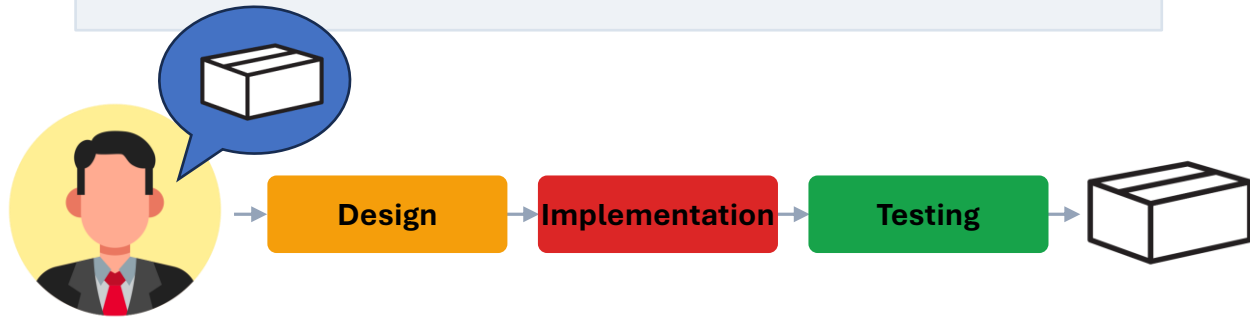
Engineers remain responsible for AI-generated code, tests, commands, and documentation submitted under their name.

SDLC models

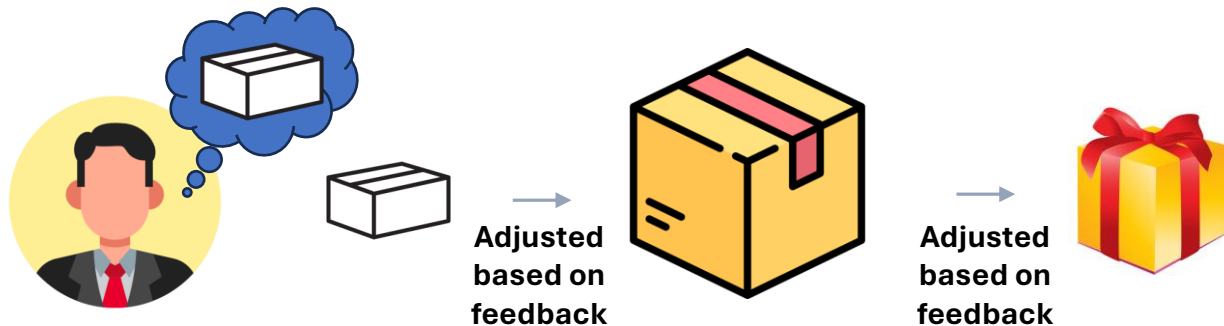
Models are ways to organize the same core activities.

Planning style

Predictive - Plan first, then execute. Change is controlled.

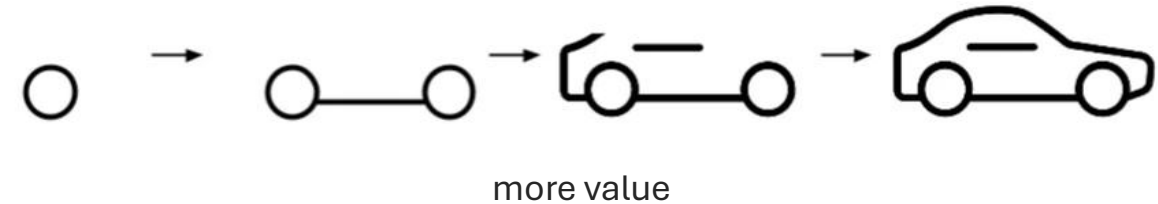


Adaptive - Learn while building. Change is expected.



Delivery style

Incremental - Deliver working slices of the product.



Iterative - Refine the same solution through feedback cycles.



Examples of SDLC models - Waterfall



Use

- Requirements are stable and well understood.
- Team has experience building similar software
- Define change-control rules before implementation begins.

Pros

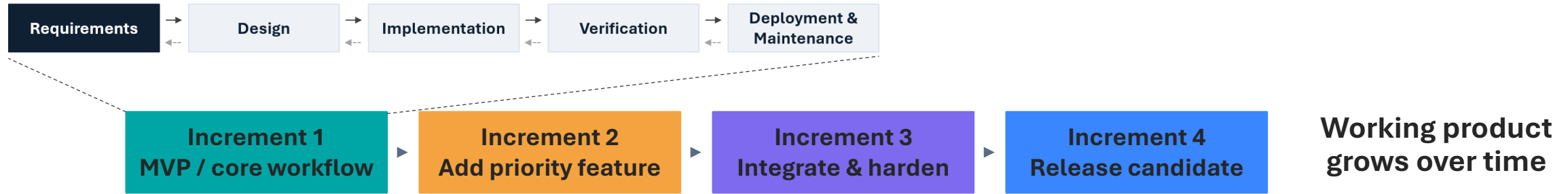
- Clear milestones and responsibilities.
- Strong traceability from requirement to release.
- Easier upfront budgeting and scheduling.

Cons

- First release takes a long time.
- Feedback arrives late.
- Changes become expensive (not flexible for changes).

Waterfall is not bad; it fits low-uncertainty work better than discovery-heavy work.

Examples of SDLC models – Incremental model



Use

- If organization may benefit from early delivery of part of product.
- If building one increment will help define future increments.

Pros

- Early user feedback & make necessary changes between increments.
- Earlier business value.
- Better risk management than one big release.

Cons

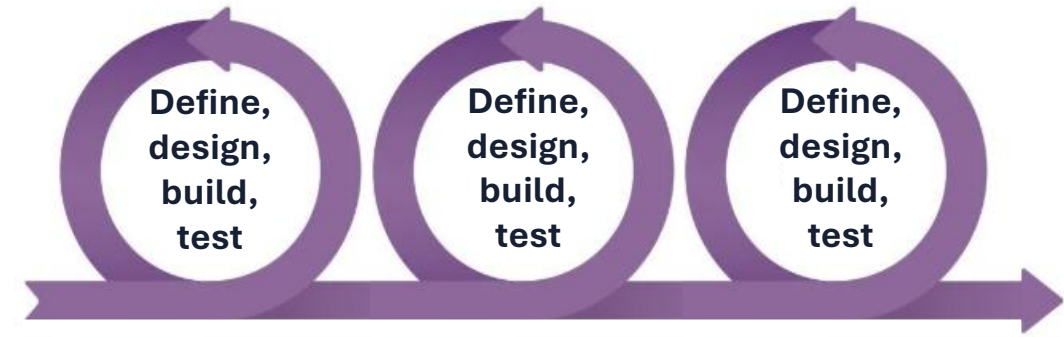
- Needs disciplined architecture and integration.
- May result in rework.
- Requires continuous validation.

Increment $\neq$ unfinished fragment. Each increment should be coherent enough to evaluate.

Examples of SDLC models – Agile values

Agile emphasizes feedback, adaptation, and working software in increments.

Agile Alliance was officially formed in 2001. Starting with Manifesto to principles.



Individual & interaction over processes & tools

Effective communication among team members.

Working software over comprehensive documentation

Regular delivery of working software allows early validation and feedback from users.

Customer collaboration over contract negotiation

Customers provide regular feedback, ensuring the product aligns with their expectations.

Responding to change over following a plan

Flexible planning ensures the product remains relevant and up to date.

People & interaction

Pros – early detection

Cons – requires participation

Adaptive

Pros – early detection

Cons –

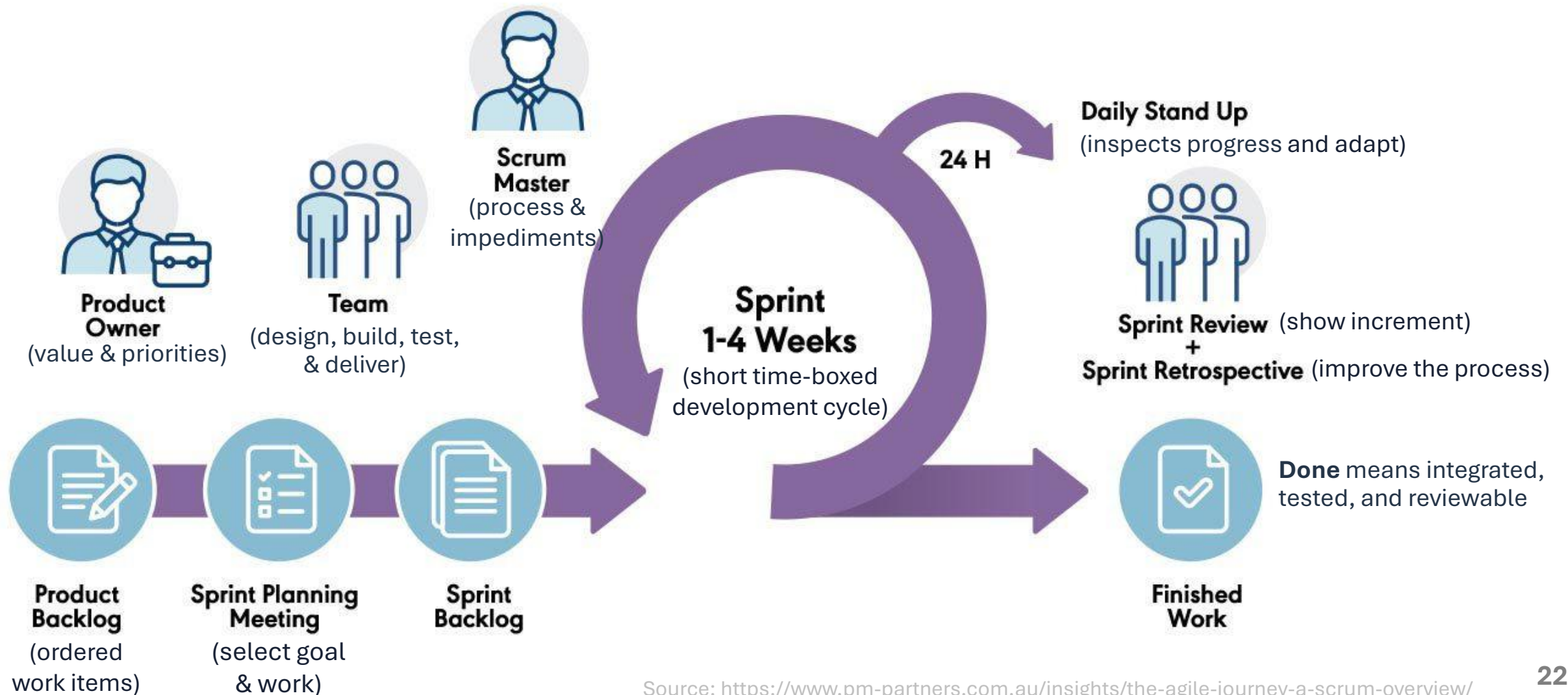
System modelling is challenging.

Lack of control.

Examples of SDLC models – Agile frameworks

Scrum

Choose Scrum when product discovery benefits from sprint goals and regular ceremonies.



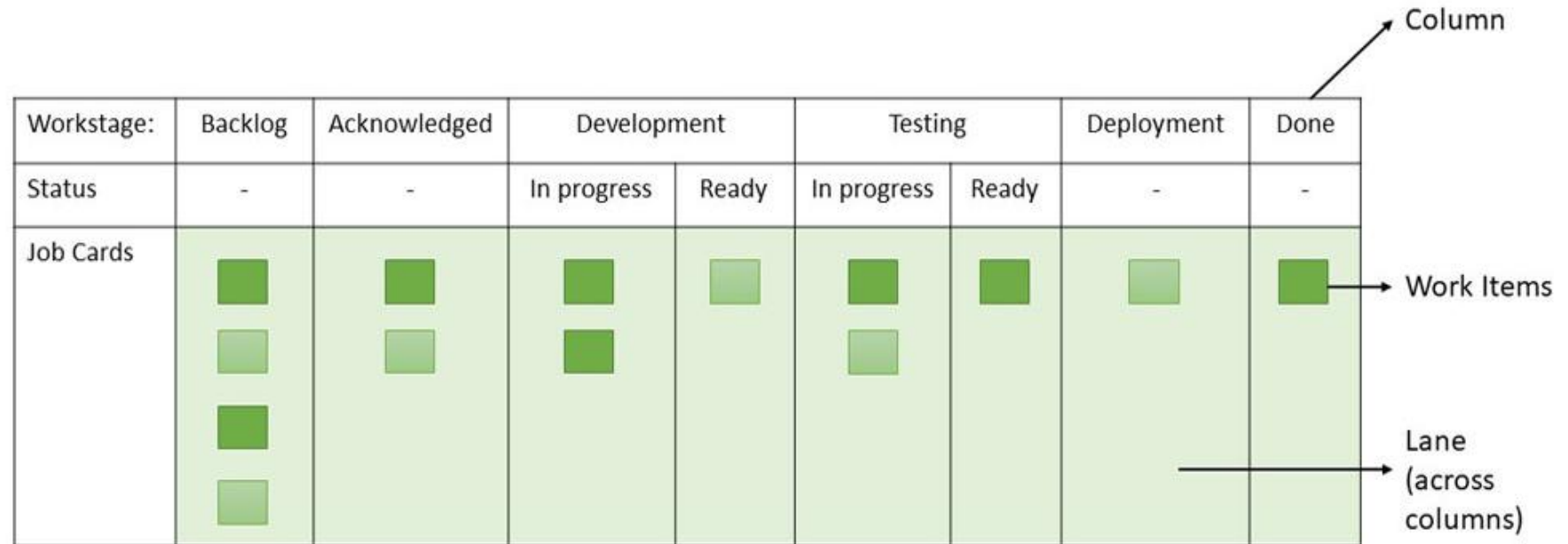
Examples of SDLC models – Agile frameworks

Kanban

Choose Kanban when work arrives continuously and controlling flow is the main problem.

Flow-management system

- Visualize workflow
- Limit work in progress
- Manage flow
- Make policies explicit
- Improve collaboratively



Source: <https://www.geeksforgeeks.org/software-engineering/kanban-agile-methodology/>

- Work Item (Type 1)
- Work Item (Type 2)

Notes

- Cycle time: start to finish
- Throughput: completed items per period
- Blocked work: visible delays

Definition of Done: the completion gate

Done is not a feeling. It is an agreed quality checklist.

Definition of Done (DoD) = team-level conditions that every completed increment must satisfy.



Acceptance criteria
satisfied



Code reviewed



Tested
(are written and passed)



No critical defects/security
issues



Docs / comments updated



Merged/deployed according
to policy

The DoD is used as a gate at the end of a story/increment/sprint or release after implementation and verification, before calling work complete.

Lecture takeaway: AI-native, engineering-first

The goal is not to avoid AI. The goal is to use AI inside a reliable engineering process.

Product thinking

users, value, scope,
acceptance

Engineering process

SDLC, design, tests, DoD

AI agent skill

delegation, review,
debugging

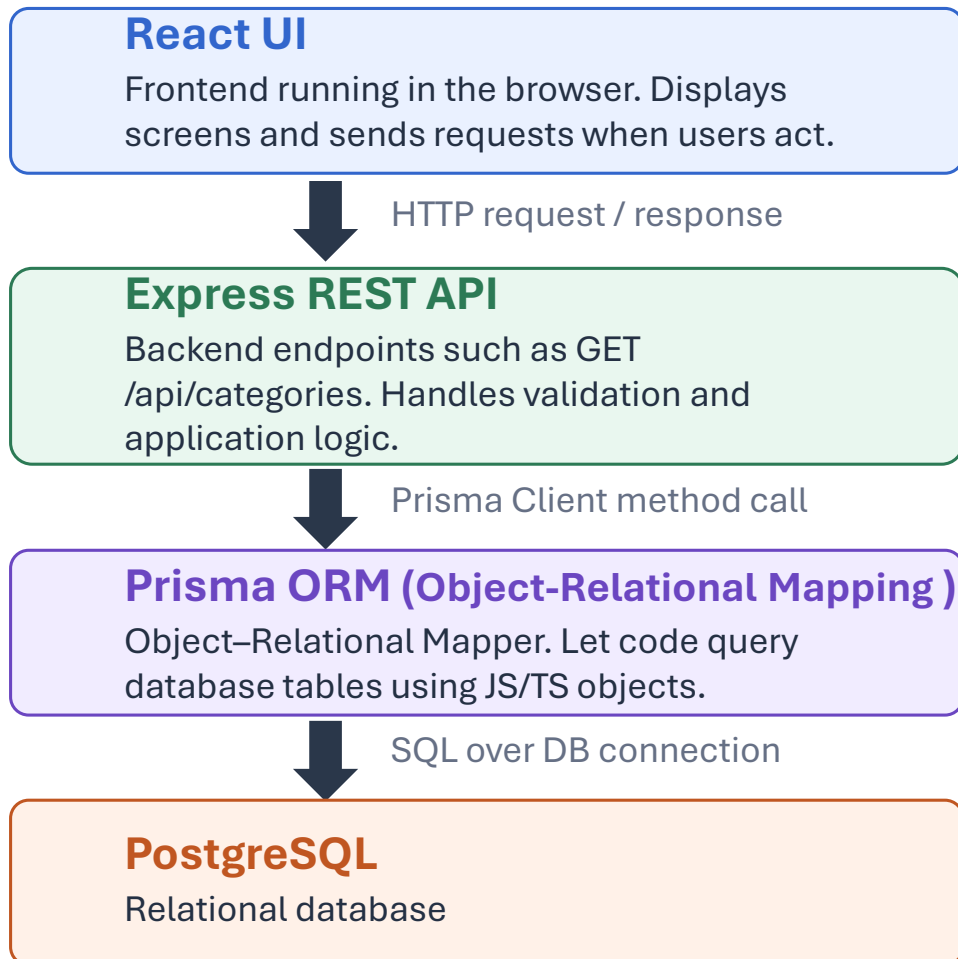
Good software engineers in the age of agents can define the work, constrain the agent, verify the result, and explain the trade-offs.

Let's prepare for our lab session.

Full-stack web app: from UI to database

A full-stack app is a set of layers that work together: the user interface, server logic, data access layer, and persistent storage.

Typical Lab Architecture



Frontend

User-facing code in the browser: screens, forms, interaction, client-side state.

Backend

Server-side code: business rules, authentication checks, validation, data access.

API

The contract between frontend and backend: endpoint, method, input, output, status codes.

ORM

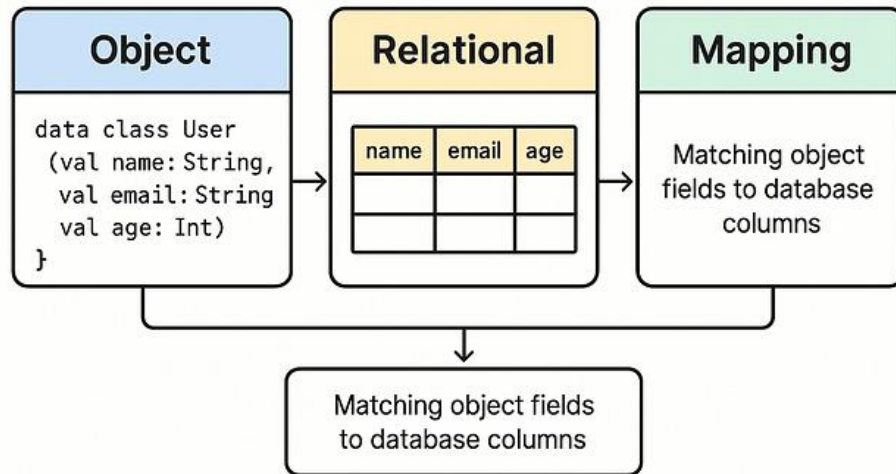
A programming layer that maps objects/classes to database tables and creates SQL queries.

Database

The persistent source of truth for structured data and relationships.

ORM idea

The developer writes object-style code; the ORM generates database queries and maps results back into objects.

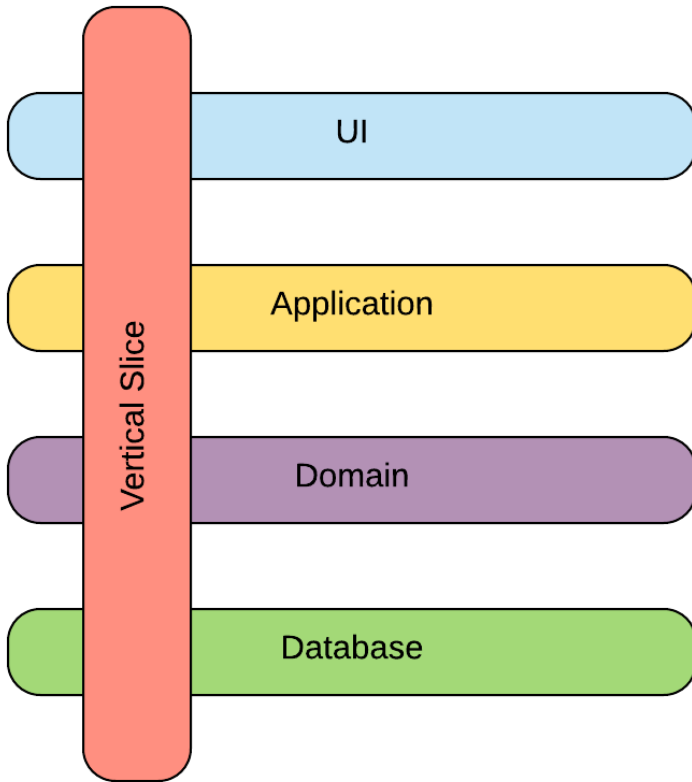


Source: <https://medium.com/@lordpacific/what-is-orm-a-beginners-guide-to-object-relational-mapping-0a20707fcdd1>

```
</> prisma
model User {
  name  String
  email String   @Id
  age   Int
  @@map("user")
}
```

```
</> Typescript
const users = await prisma.user.findMany();
//SELECT * FROM user;
```

Vertical slice



Source: <https://www.gianty.com/vertical-slice-game-development/>

- A project management technique
- Emphasizes delivering a fully functional piece of a project, encompassing all layers of the application stack.
- Ensures that each feature or functionality is developed and delivered end-to-end in a single iteration.

Specification-Driven and Test-Driven Development

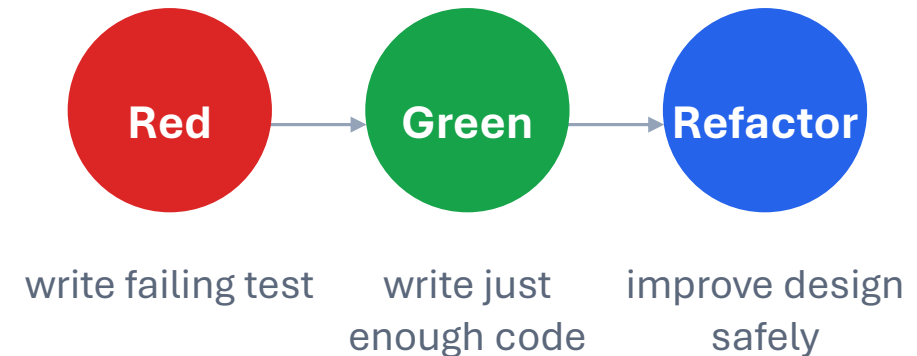
Two disciplines that make AI-assisted coding safer.

Specification-Driven Development

Start by writing the expected behavior clearly enough that implementation choices can be judged.

Typical artifacts: user story, specification, API contract, examples, acceptance criteria.

Test-Driven Development



With agents: write the spec → ask the agent to derive tests → review tests → implement → verify against the spec.

Version control

**Track changes to
source code**

who, what, when, why

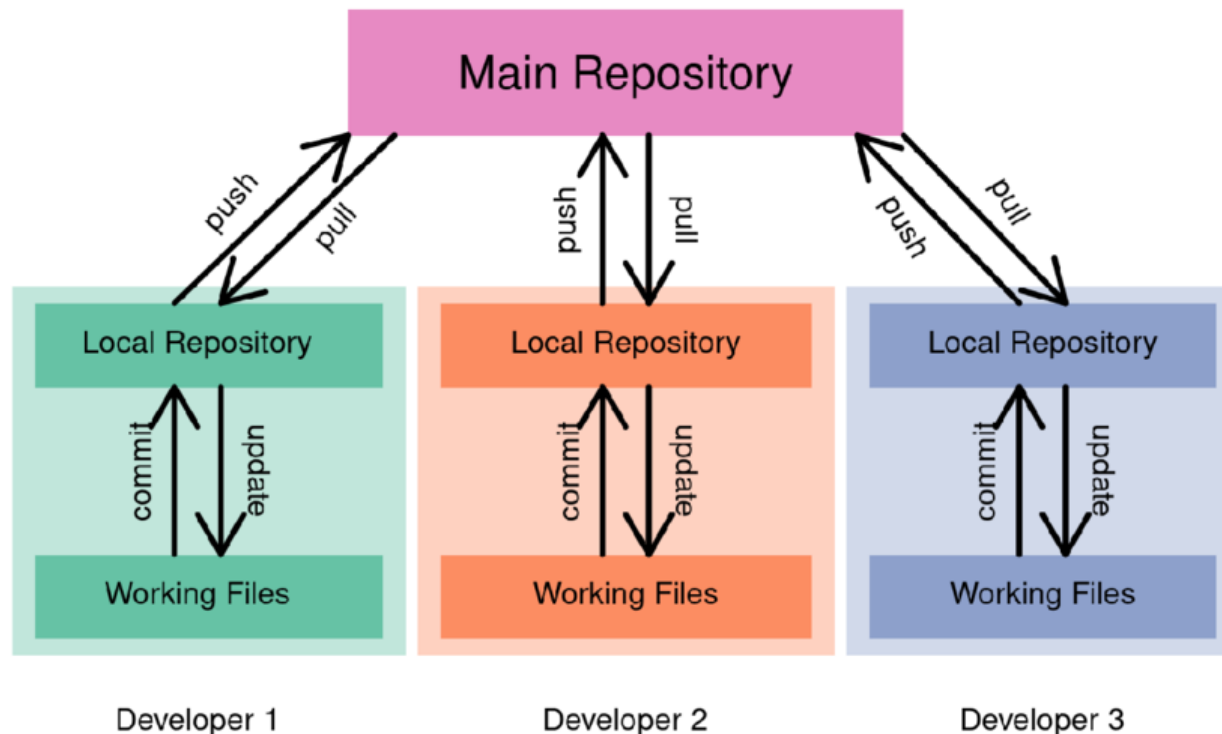
**Recovers older
version**

before experiments and
agent edits

**Facilitate
collaboration**

avoid code conflicts &
overwriting work

Version control system (VCS) - a software tool to track & manage changes to source code & digital files



Distributed VCS



git

Command line

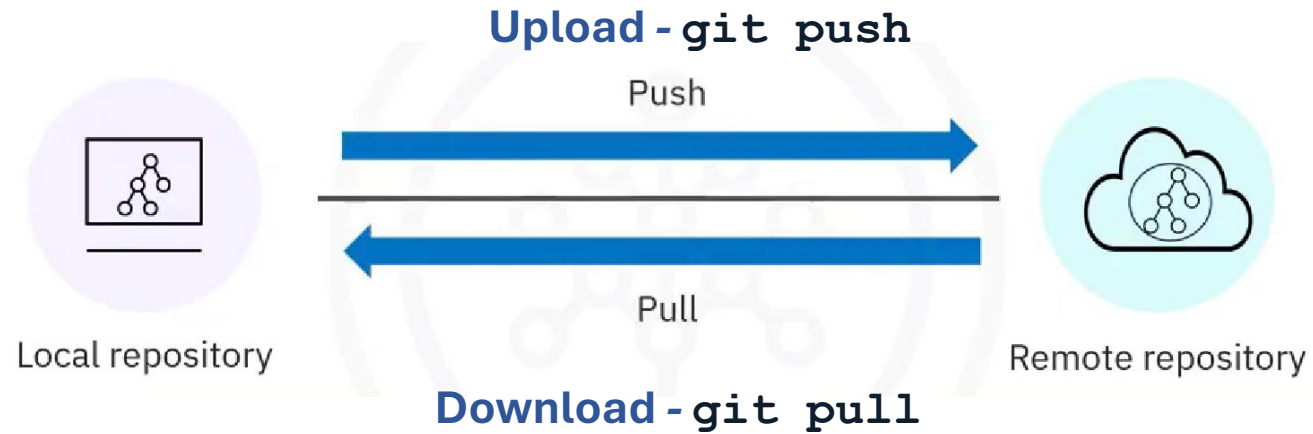


GitHub

Web-hosted services

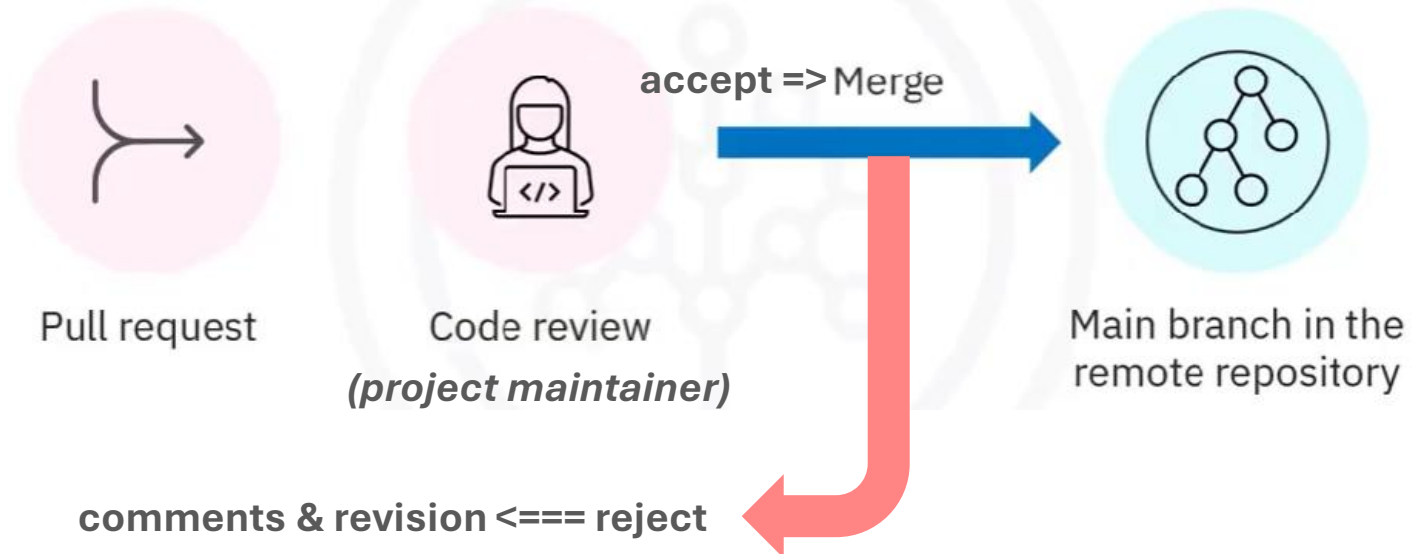
Git workflow

Repositories



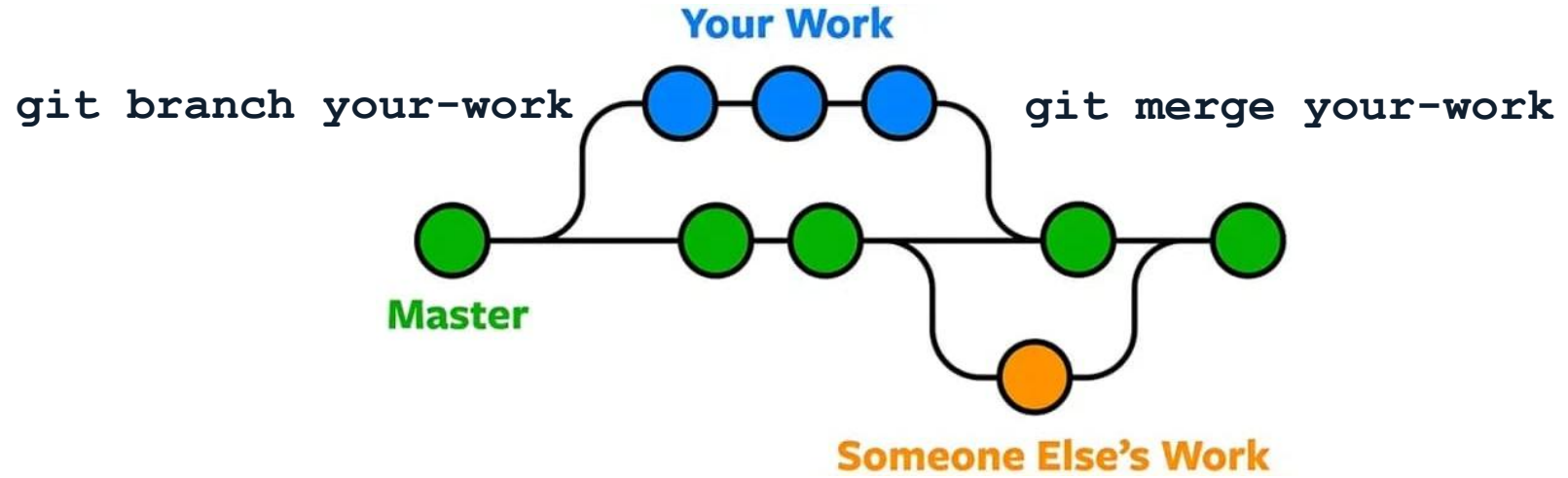
Checking that it all makes sense & follows the expected patterns

Reviewing code (Github)



Git workflow

Branch

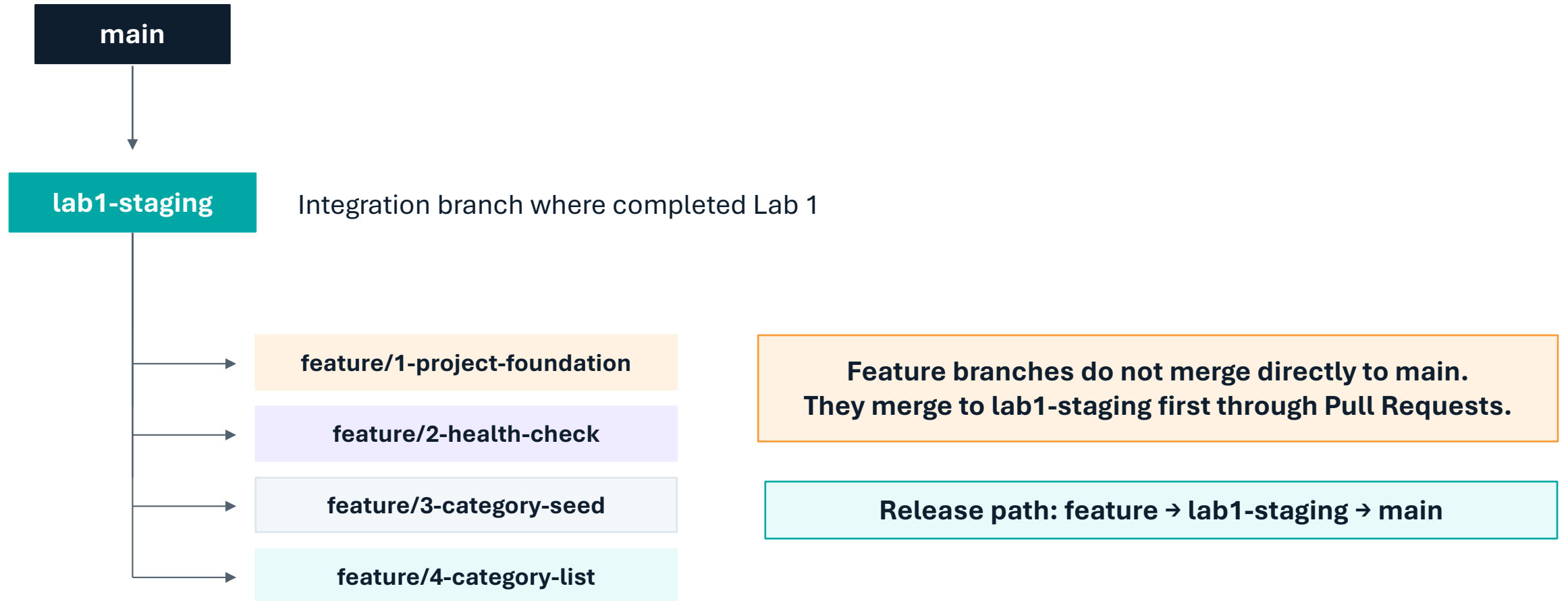


Source: <https://novatorsoft.com/en/blog/what-is-branch-how-to-use>

Main (master): stable release branch. Code should represent accepted output.

Feature branches: short-lived branches for one small feature.

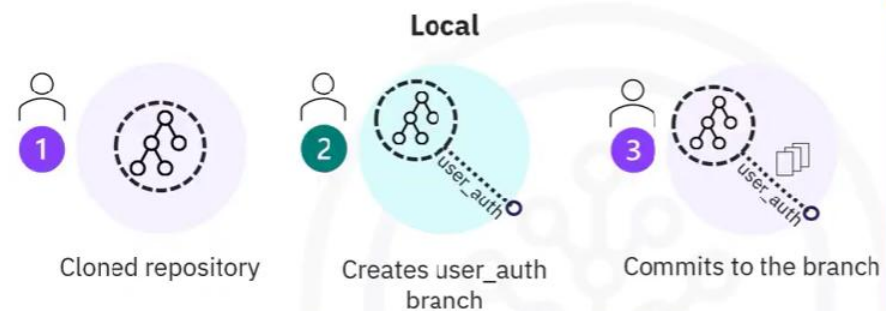
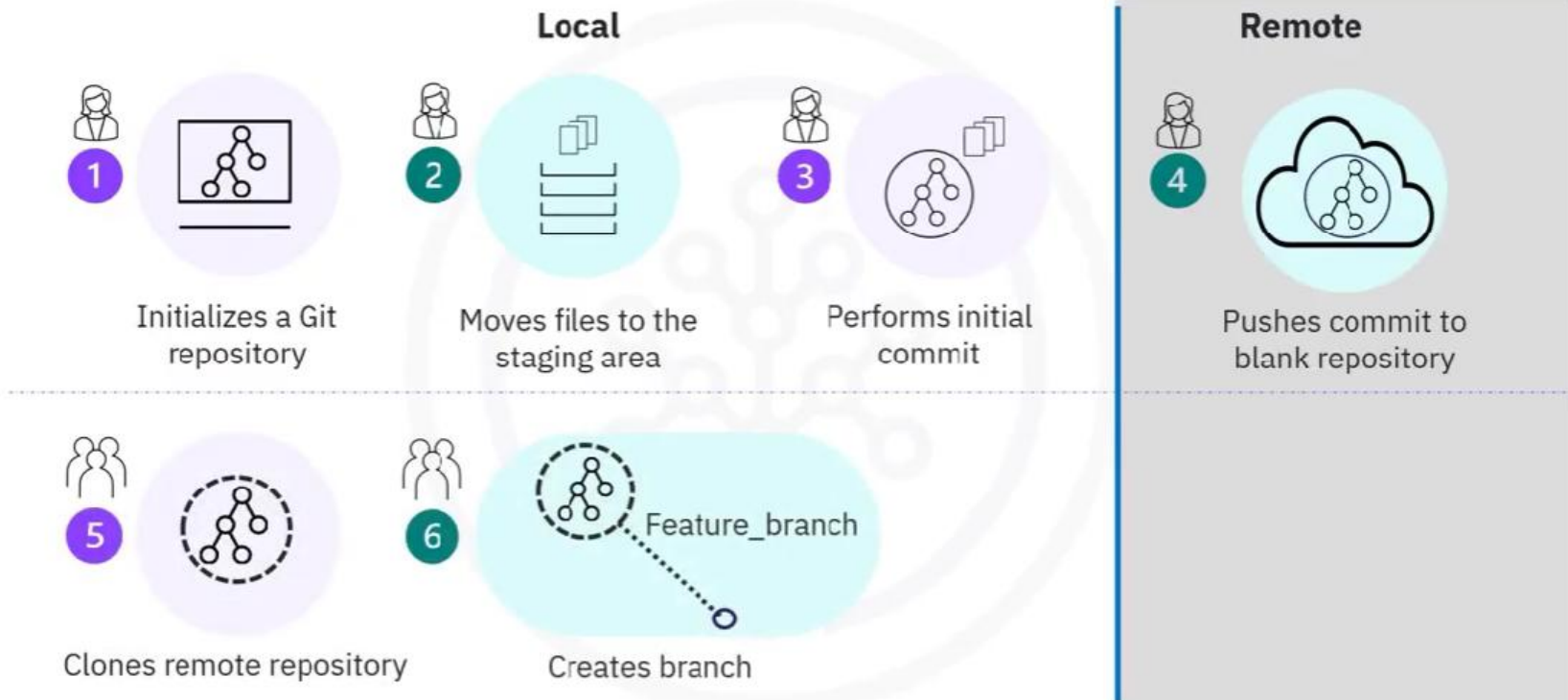
Git workflow



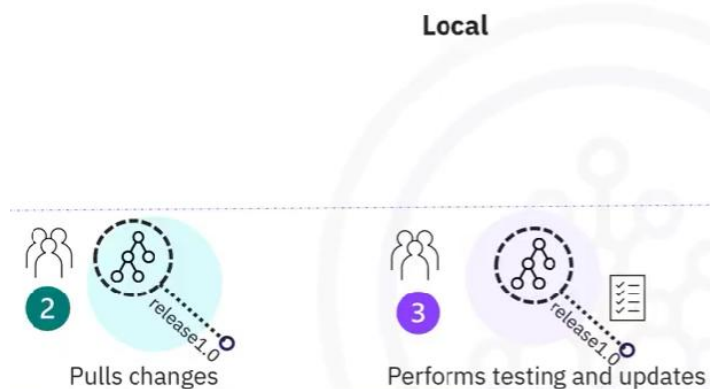
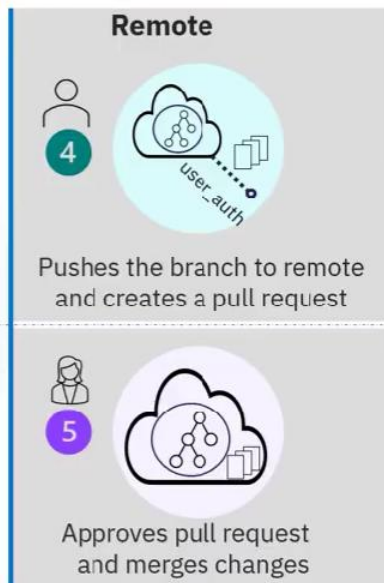
Git workflow

Scenario: web app development

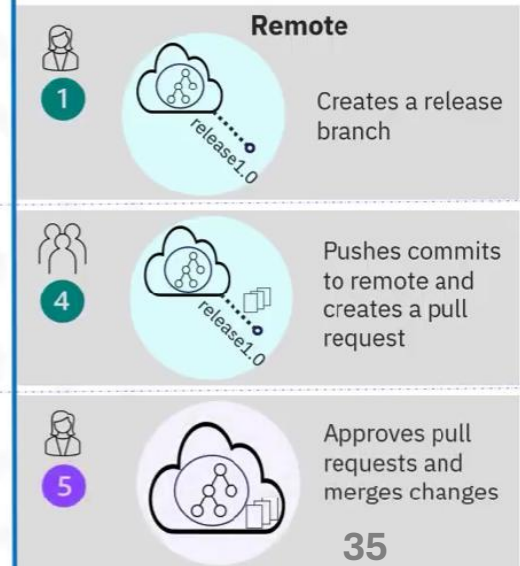
Legend:  Lead developer  Team



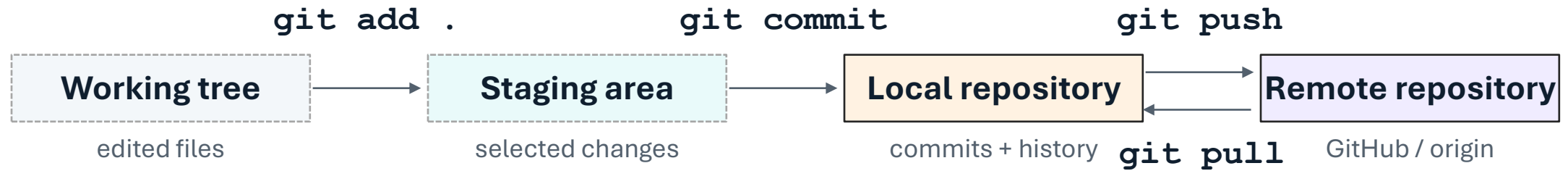
Feature development



Project release



Git workflow



```
# Select changes to include in the next checkpoint
git add README.md src/app.js

# Save the checkpoint in local commit history
git commit -m "Add project foundation"

# Share local commits with GitHub
git push origin feature/1-project-foundation
```

```
# Update the current local branch from its remote branch
git pull origin lab1-staging

# Inspect history
git log --oneline --graph -decorate

# Show the current state of working directory & staging area
git status
```



Quick recap

Repository	A project folder tracked by Git.	Working tree	Files you are currently editing.
Remote repository	The repo hosted on GitHub.	Staging area	Changes selected for the next commit.
Local repository	The repo on your computer.	Commit	A saved snapshot of the project's current state.
Origin	The usual remote name for GitHub.	Commit history	The ordered record of snapshots.
Clone	Local copy of the remote Git repository	Branch	A named, separate line of development.
Merge	Combine changes from one branch to another.		

Github overview

Online hosting service for Git repositories

The screenshot shows the GitHub interface for a repository named 'se-demo' owned by 'piyanitwep-sys'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main', and there are 2 branches and 0 tags. A recent commit is shown: 'Merge pull request #1 from piyanitwep-sys/piyanitwep-sys-patch-1' by 'piyanitwep-sys' 7 hours ago, with 3 commits. The file 'README.md' is highlighted, showing an update 7 hours ago. The README content includes the title 'se-demo' and the text 'For simple demo how to use Github Edit message'. The right sidebar contains sections for 'About' (For simple demo how to use Github), 'Releases' (No releases published, with a link to 'Create a new release'), 'Packages' (No packages published, with a link to 'Publish your first package'), and 'Contributors' (1 contributor, 'piyanitwep-sys').

piyanitwep-sys / se-demo

Code Issues 1 Pull requests Actions Projects Wiki Security and quality Insights Settings

se-demo Public

Pin Watch 0 Fork 0 Star 0

main 2 Branches 0 Tags

Go to file Add file Code

piyanitwep-sys Merge pull request #1 from piyanitwep-sys/piyanitwep-sys-patch-1 774cfe · 7 hours ago 3 Commits

README.md Update README.md 7 hours ago

README

se-demo

For simple demo how to use Github Edit message

About

For simple demo how to use Github

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

piyanitwep-sys

Github code review

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

Landing Page Improvements #1

Merged mgreiler merged 27 commits into main from 2022/12/landing-page-improvements last month

Conversation 1 Commits 27 Checks 5 Files changed 98 +2,439 -218

Changes from 1 commit File filter Conversations Settings

Filter changed files

src

components

Navigation.js

layout.js

nav.css

style.css

Adapt overall layout, nav, and footer

main (#2)

LenaSchnedlitz committed on Dec 27, 2022

commit a6ee957249bed4cac2be477776ebcc60dd6f25a7

118 src/components/Navigation.js

@@ -1,106 +1,42 @@

1 - import * as React from "react"

2 import { Link } from "gatsby"

3 - import "../nav.css"

4 import { StaticImage } from "gatsby-plugin-image"

5 - import { Nav, Navbar, Container } from "react-bootstrap"

6

7 const Navigation = ({ location }) => {

8 - let header

9 -

10 header = (

11 - <nav className="navbar-brand title-h1">

12 <Link to="/">

13 <StaticImage

1 import { Link } from "gatsby"

2 import { StaticImage } from "gatsby-plugin-image"

3 + import * as React from "react"

4 + import { Nav, Navbar } from "react-bootstrap"

5

6 const Navigation = ({ location }) => {

7 + return (

8 + <Navbar bg="white" expand="lg" className="content">

9 <Link to="/">

10 <StaticImage

Source: <https://www.awesomecodereviews.com/tools/best-code-review-tools/>

Github issue & kanban



Issues

Code | **Issues 2,798** | Pull requests 578 | Actions | Projects 5 | Security | Insights

Filters | is:issue is:open | Labels 169 | Milestones 14 | **New issue**

2,798 Open | 17,669 Closed

Author | Label | Projects | Milestones | Assignee | Sort

- Testing: Travis Integration out-of-date `comp: docs`
#36451 opened 42 minutes ago by gklandes | needsTriage
- Components are not removed from DOM when calling `NgModuleRef.destroy()` `comp: core`
`comp: ivy` `core: DOM rendering` `core: NgModule` `severity: regression`
#36449 opened 2 hours ago by arturovt | needsTriage
- Cannot use the service worker in AWS CloudFront with `PathLocationStrategy`
#36439 opened 18 hours ago by stefan-karlsson
- ngControl is Undefined using with directive after update to Angular 9 IVY
#36438 opened 19 hours ago by ofirifo
- Angular Material Not working after update(release 9.0.5) on 4-4-2020 `state: Needs repro`
`type: bug/fix`
#36435 opened yesterday by 4-TREX
- Detect attributes used in deps for providers `comp: language-service`
#36433 opened 2 days ago by waterplea | needsTriage
- Unit test should fail if an element is not known `comp: core` `comp: ivy` `comp: testing`
`core: directive matching` `freq3: high` `severity: regression` `type: bug/fix`

Note:

Issue: describes work, acceptance criteria, dependencies & evidence needed.

Branch: isolates code changes that implement one Issue.

Kanban

Repositories | People 10 | Teams 5 | **Projects 4** | Settings

Sample 114h 32m · 3h 20m

Filter cards

Backlog 4 8h 19m

- Fullscreen mode refactoring
everhour/everhour-web#624 opened by yurytolochko
frontend
- JSON: discriminator
everhour/everhour-framework#13 opened by yurytolochko
- Migrate to recent angular2 version
everhour/everhour-web#640 opened by yurytolochko
frontend
- Data refactoring
0 of 2
everhour/everhour-browser-extension#167 opened by yurytolochko
8h 19m
extension

Planned 2 14h 27m

- Stop timer from team dashboard
8 of 9
everhour/everhour-web#643 opened by yurytolochko
14h 27m
backend frontend
- Task properties (asana custom fields)
0 of 7
everhour/everhour-web#649 opened by yurytolochko
backend frontend

In Progress 3

- #647 Summary emails
everhour/everhour-web#672 opened by yurytolochko
1100:00:20
in progress
- #660 Invoice localization
everhour/everhour-web#670 opened by sergei-dzekevich
in progress
- #644 Autostop timer
everhour/everhour-web#668 opened by yurytolochko
in progress

Done 7 91h 46m · 3h 20m

- Highlight overtime
everhour/everhour-browser-extension#172 opened by yurytolochko
4h 31m
- GitHub boards
everhour/everhour-browser-extension#171 opened by sergei-dzekevich
10h 53m
- Install Wootric (NPS score tool)
everhour/everhour-web#632 opened by citizenblr
- Email changes
everhour/everhour-api#435 opened by yurytolochko
1h 21m of 3h 20m
- Jira support
everhour/everhour-browser-extension#160 opened by sergei-dzekevich
67h 26m
- Total time of my tasks in Asana are missing
everhour/everhour-browser-extension#161 opened by wacław-włodko
bug

Repository setup

Case 1 – clone from Github

```
# 1) Copy the repository URL from GitHub
# Example: https://github.com/org/se-ai-lab1.git

git clone https://github.com/org/se-ai-lab1.git
cd se-ai-lab1

# 2) Check where you are and what branch is active +
change branch
git status
git branch
git checkout -b new-branch

# 3) Inspect the latest commits
git log --oneline --decorate -5
```

```
# Verify remote location
git remote -v

# Clean start should show:
# nothing to commit, working tree clean
git status
```

Case 2 – initialize a safe new one

```
mkdir se-ai-lab1
cd se-ai-lab1

git init

# Create basic project files
echo "# SE AI Lab 1" > README.md
echo "node_modules/" > .gitignore
echo ".env" >> .gitignore
echo "DATABASE_URL=" > .env.example

# First local commit
git add README.md .gitignore .env.example
git commit -m "Initial project setup"
```

```
# Connect to GitHub and push main
git branch -M main
git remote add origin https://github.com/org/se-ai-lab1.git
git push -u origin main
```

The commit message explains the engineering purpose, not just say “update files”.

Repository setup

Never commit

- .env files
- passwords or API keys
- database credentials
- node_modules/
- generated build files
- large temporary outputs

README should state

- What the project is
- How to install and run
- Required environment variables
- How to run tests
- Branch and Pull Request (PR) rules for the lab

Create a feature branch – push – open PR

Example Issue: #1 Project foundation

```
# 0) Get a list of branch
git branch

# 1) Start from the integration branch
git switch lab1-staging

# 2) Get the latest staging work from GitHub
git pull origin lab1-staging

# 3) Create a feature branch for one Issue
git switch -c feature/1-project-foundation

# 4) Work, inspect, stage, and commit
git status
git add .
git commit -m "Set up project foundation"

# 5) Publish the branch to GitHub for the first time
git push -u origin feature/1-project-foundation
```

Note

- Starting from lab1-staging avoids building on outdated code.
- The branch name connects the code change to a specific Issue.
- Small commits make review and debugging easier.

Then in GitHub

- Create PR from feature/1-project-foundation into lab1-staging.
- Link the Issue in the PR description, for example: “Closes #1”.
- Paste acceptance criteria and mark what is satisfied.
- Request peer review before merging.

Before asking an AI agent to implement, create the branch first so its changes are isolated.

Keep your feature branch current before merge

Merge another feature into lab1-staging first

```
# 1) Download latest remote information
git fetch origin

# 2) Stay on your feature branch
git switch feature/2-health-check

# 3) Merge the latest staging branch into your feature branch
git merge origin/lab1-staging

# 4) If conflicts happen: edit files, then
git status
git add <resolved-files>
git commit

# 5) Update the Pull Request
git push
```

```
# Optional local cleanup after merge
git switch lab1-staging
git pull origin lab1-staging
git branch -d feature/2-health-check
```

After PR approval

- Merge into lab1-staging, not directly into main.
- Delete completed remote feature branches after merge.
- Keep local branches tidy after they are finished.

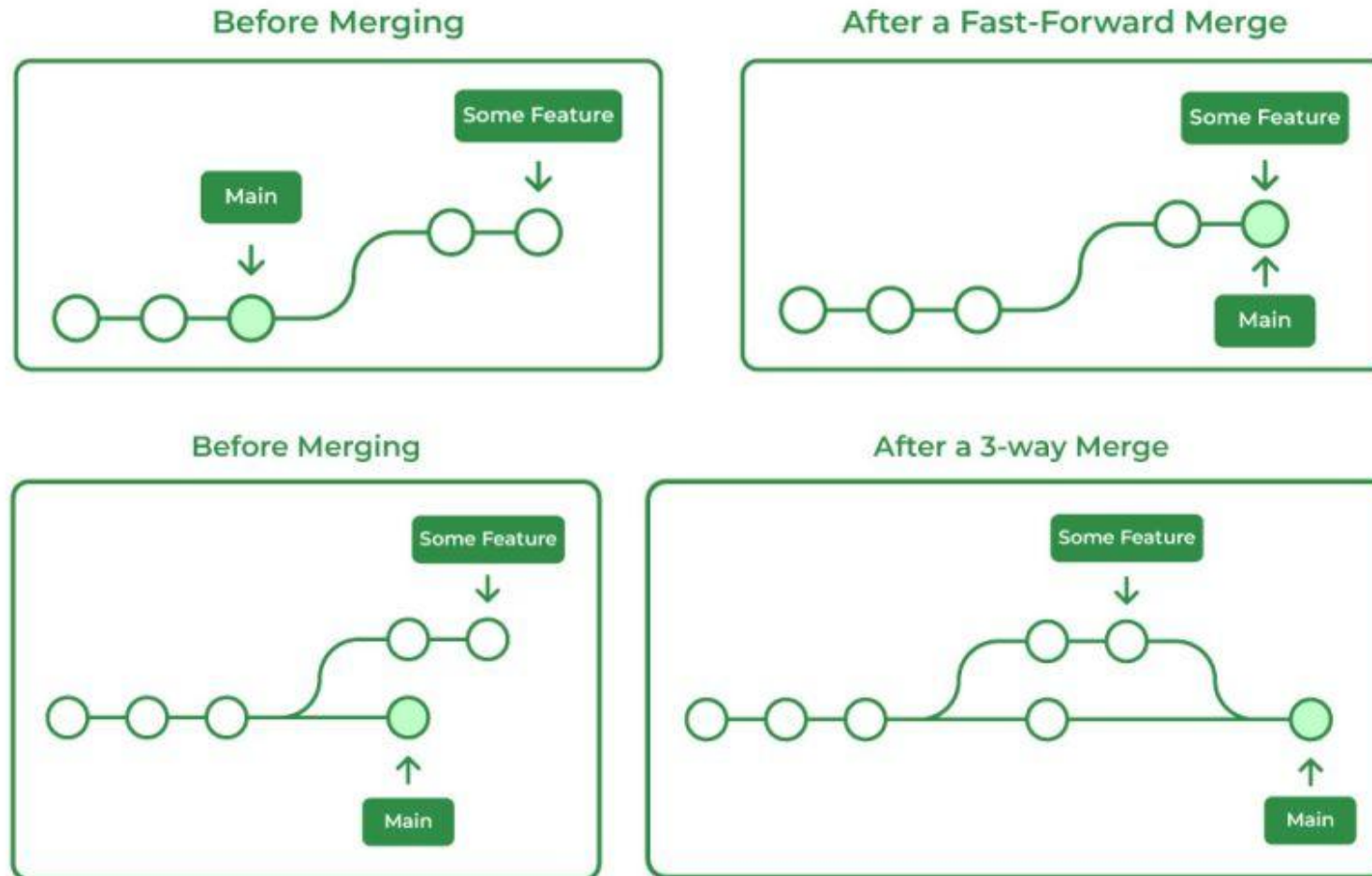


Local repository



Remote repository

Fast-forward merge vs 3-way merge



Source: https://www.linkedin.com/posts/aftabnajib_git-branching-mergeconflicts-activity-7359217391152603138-L5lt/

Release Lab 1 from staging to main



- Confirm all Lab 1 feature PRs are merged into lab1-staging.
- Run required checks or demonstrations from lab1-staging.
- Create a PR from lab1-staging into main.
- Merge with a merge commit after approval.

Release means the integrated lab work is accepted on main, not merely that individual features were pushed.

Done = accepted by review + satisfies criteria + merged + evidence documented
≠ the AI agent said it works